

LƯU Ý: Viết API Nhóm 6 — Products Public

Dành cho đặc vụ AI trong VS Code (Claude Code / Copilot / Cursor)

MỤC TIÊU

Viết 3 API thuộc nhóm “**6. Products - Public**” cho dự án backend thương mại điện tử TechTonicEcommerce_Nhóm2.

TRƯỚC KHI VIẾT BẤT KỲ DÒNG CODE NÀO , hãy đọc toàn bộ codebase hiện tại để hiểu đúng kiến trúc, quy ước và mẫu đang được sử dụng.

BƯỚC 1: KHÁM PHÁ CODEBASE HIỆN TÍCH (BẮT BUỘC)

Thực hiện các lệnh sau và ghi kết quả ghi nhớ:

đáp

```
# 1. Xem cấu trúc thư mục tổng thể
find . -type f -name "*.js" -o -name "*.ts" | grep -v node_modules | sort

# 2. Đọc package.json để biết framework, thư viện, scripts
cat package.json

# 3. Đọc file entry point
cat src/index.js # hoặc src/app.js hoặc src/server.js

# 4. Xem cấu trúc routes đã có
ls src/routes/

# 5. Đọc 1 file route đã hoàn thành (ưu tiên auth hoặc categories)
cat src/routes/auth.routes.js # hoặc tên file tương ứng
cat src/routes/category.routes.js

# 6. Đọc controller tương ứng để hiểu pattern
cat src/controllers/auth.controller.js
cat src/controllers/category.controller.js

# 7. Xem cách query database (service hoặc model layer)
cat src/services/auth.service.js # hoặc
cat src/models/category.model.js # tùy theo pattern dự án

# 8. Đọc middleware JWT
cat src/middlewares/auth.middleware.js # hoặc tên tương ứng

# 9. Xem response format chuẩn (helper/utils)
cat src/utils/response.js # hoặc src/helpers/response.js

# 10. Đọc file cấu hình DB
cat src/config/db.js # hoặc src/database/connection.js
```

Ghi chú cho AI: Sau khi đọc, hãy tắt ra:

- Framework & phiên bản đang được sử dụng (Express, Fastify, ...)
- Cách tổ chức thư mục (tuyến đường / bộ điều khiển / dịch vụ, tuyến đường / bộ điều khiển / mô hình, ...)
- Mẫu từ cơ sở dữ liệu (truy vấn thô mssql, Sequelize, TypeORM, ...)
- Phản hồi định dạng thành công và lỗi
- Quy ước đặt tên file, hàm, biến
- Cách viết xác thực phân mềm trung gian

BƯỚC 2: THÔNG TIN CƠ SỞ DỮ LIỆU (SQL SERVER)

Các bảng liên kết đến nhóm 6:

Bang **products**

```
product_id    BIGINT IDENTITY PK
store_id      BIGINT NOT NULL → FK → stores.store_id
category_id   BIGINT NOT NULL → FK → categories.category_id
sku           NVARCHAR(50) UNIQUE (per store)
product_name  NVARCHAR(200) NOT NULL
slug          NVARCHAR(191) UNIQUE
description   NVARCHAR(MAX)
price         DECIMAL(18,2) NOT NULL CHECK (price > 0)
stock_quantity INT DEFAULT 0 CHECK >= 0
sold_quantity INT DEFAULT 0 CHECK >= 0
status        VARCHAR(20) CHECK IN ('ACTIVE','INACTIVE','SUSPENDED')
              DEFAULT 'ACTIVE'
created_at    DATETIME2(0)
updated_at    DATETIME2(0)
deleted_at    DATETIME2(0) NULL -- soft delete
```

Bang **product_images**

```
product_image_id BIGINT IDENTITY PK
product_id        BIGINT NOT NULL → FK → products.product_id
image_url         NVARCHAR(500) NOT NULL
alt_text          NVARCHAR(255)
display_order     INT DEFAULT 0
is_primary        BIT DEFAULT 0 -- UNIQUE INDEX: 1 ảnh primary mỗi product
created_at        DATETIME2(0)
updated_at        DATETIME2(0)
```

Bang **stores**

```

store_id          BIGINT IDENTITY PK
store_code        VARCHAR(30) UNIQUE
store_name        NVARCHAR(150) NOT NULL
slug              NVARCHAR(191) UNIQUE
description       NVARCHAR(MAX)
logo_url          NVARCHAR(500)
contact_email     NVARCHAR(255)
contact_phone     VARCHAR(20)
address_line      NVARCHAR(255)
ward              NVARCHAR(100)
district          NVARCHAR(100)
province          NVARCHAR(100)
total_products    INT DEFAULT 0
status            VARCHAR(20) CHECK IN ('ACTIVE','INACTIVE','SUSPENDED')
created_at        DATETIME2(0)
updated_at        DATETIME2(0)
deleted_at        DATETIME2(0) NULL

```

Bangcategories

```

category_id       BIGINT IDENTITY PK
parent_category_id BIGINT NULL → FK → categories.category_id (self-ref)
category_name     NVARCHAR(150) NOT NULL
slug              NVARCHAR(191) UNIQUE
description       NVARCHAR(500)
icon_url          NVARCHAR(500)
display_order     INT DEFAULT 0
status            VARCHAR(20) CHECK IN ('ACTIVE','INACTIVE')
created_at        DATETIME2(0)
updated_at        DATETIME2(0)
deleted_at        DATETIME2(0) NULL

```

Bangreviews

```

review_id          BIGINT IDENTITY PK
order_item_id      BIGINT NOT NULL → FK → order_items.order_item_id (UNIQUE:
1 review/item)
product_id         BIGINT NOT NULL → FK → products.product_id
customer_id        BIGINT NOT NULL → FK → users.user_id
rating            TINYINT NOT NULL CHECK BETWEEN 1 AND 5
comment            NVARCHAR(1000) NULL
status            VARCHAR(20) CHECK IN ('VISIBLE','HIDDEN') DEFAULT
'VISIBLE'
hidden_by_user_id  BIGINT NULL
hidden_at          DATETIME2(0) NULL
created_at         DATETIME2(0)
updated_at         DATETIME2(0)

```

Bảng `users` (chỉ cần để THAM GIA lấy thông tin người đánh giá)

```

user_id    BIGINT IDENTITY PK
full_name  NVARCHAR(150)
avatar_url NVARCHAR(500) NULL

```

BƯỚC 3: ĐẶC TẢ 3 API CẦN VIẾT

API #18 — Liệt kê sản phẩm

Cánh đồng	Giá trị
-----------	---------

Phương pháp	LẤY
-------------	-----

Điểm cuối	<code>/api/v1/products</code>
-----------	-------------------------------

Xác thực	Không bắt buộc (Công khai)
----------	----------------------------

Mô tả	Danh sách sản phẩm khai báo, bộ lọc hỗ trợ và phân trang
-------	--

Tham số truy vấn:

page	INT	Trang hiện tại (default: 1, min: 1)
limit	INT	Số item mỗi trang (default: 20, max: 100)
keyword	STRING	Tìm kiếm theo product_name (LIKE %keyword%)
category_id	INT	Lọc theo category_id
store_id	INT	Lọc theo store_id
sort_by	STRING	Trường sắp xếp: 'price' 'sold_quantity' 'created_at' (default: 'created_at')
sort_order	STRING	'ASC' 'DESC' (default: 'DESC')
min_price	DECIMAL	Lọc giá từ
max_price	DECIMAL	Lọc giá đến

Quy tắc kinh doanh:

- Chỉ trả về sản phẩm có `status = 'ACTIVE'` và `deleted_at IS NULL`
- Chỉ trả về sản phẩm của cửa hàng và `status = 'ACTIVE'` có `deleted_at IS NULL`
- `sort_by` chỉ chấp nhận giá trị trong danh sách trắng, nếu không hợp lệ `created_at`
- `sort_order` chỉ được chấp nhận `ASC` hoặc `DESC`, nếu không hợp lệ `DESC`
- `max_price` phải $\geq$ `min_price` nếu cả 2 được truyền tải

Mẹo truy vấn SQL (điều chỉnh theo ORM nếu được sử dụng):

SQL

```

SELECT
    p.product_id,
    p.product_name,
    p.slug,
    p.price,
    p.stock_quantity,
    p.sold_quantity,
    p.status,
    p.created_at,
    c.category_id,
    c.category_name,
    s.store_id,
    s.store_name,
    s.logo_url AS store_logo,
    img.image_url AS primary_image_url,
    ROUND(AVG(CAST(r.rating AS FLOAT)), 1) AS avg_rating,
    COUNT(DISTINCT r.review_id) AS review_count
FROM products p
INNER JOIN stores s ON p.store_id = s.store_id
    AND s.status = 'ACTIVE' AND s.deleted_at IS NULL
INNER JOIN categories c ON p.category_id = c.category_id
LEFT JOIN product_images img ON img.product_id = p.product_id AND img.is_primary
LEFT JOIN reviews r ON r.product_id = p.product_id AND r.status = 'VISIBLE'
WHERE p.status = 'ACTIVE'
    AND p.deleted_at IS NULL
    -- Filters được thêm động theo params
    -- AND p.product_name LIKE '%keyword%'
    -- AND p.category_id = @category_id
    -- AND p.store_id = @store_id
    -- AND p.price >= @min_price
    -- AND p.price <= @max_price
GROUP BY p.product_id, p.product_name, p.slug, p.price,
    p.stock_quantity, p.sold_quantity, p.status, p.created_at,
    c.category_id, c.category_name,
    s.store_id, s.store_name, s.logo_url,
    img.image_url
ORDER BY p.created_at DESC -- điều chỉnh theo sort_by + sort_order
OFFSET (@page - 1) * @limit ROWS
FETCH NEXT @limit ROWS ONLY;

-- Riêng biệt: đếm tổng để tính total_pages
SELECT COUNT(DISTINCT p.product_id) AS total ...

```

Phản hồi thành công (200):

json

```
{
  "success": true,
  "data": {
    "products": [
      {
        "product_id": 1,
        "product_name": "Áo thun basic trắng",
        "slug": "ao-thun-basic-trang",
        "price": 150000,
        "stock_quantity": 50,
        "sold_quantity": 23,
        "primary_image_url": "https://...",
        "avg_rating": 4.5,
        "review_count": 12,
        "category": {
          "category_id": 2,
          "category_name": "Áo"
        },
        "store": {
          "store_id": 1,
          "store_name": "Shop Thời Trang An",
          "store_logo": "https://..."
        }
      }
    ],
    "pagination": {
      "current_page": 1,
      "per_page": 20,
      "total_items": 85,
      "total_pages": 5
    }
  }
}
```

API #19 — Lấy thông tin chi tiết sản phẩm

Cánh đồng	Giá trị
Phương pháp	LẤY
Điểm cuối	/api/v1/products/:id
Xác thực	Không bắt buộc (Công khai)
Mô tả	Chi tiết 1 sản phẩm: thông tin đầy đủ, tất cả hình ảnh, thông tin cửa hàng, thống kê rating

Các thông số đường dẫn:

id	BIGINT	product_id (bắt buộc)
----	--------	-----------------------

Quy tắc kinh doanh:

- If `product_id` không tồn tại → 404
- If sản phẩm có `deleted_at IS NOT NULL` → 404 (coi như không tồn tại)
- If sản phẩm có `status = 'INACTIVE'` hoặc `'SUSPENDED'` → trả 404 (ẩn công khai)
- If shop of sản phẩm bị `SUSPENDED` hoặc `deleted_at IS NOT NULL` → trả 404
- Trả về **tất cả ảnh** (không chỉ ảnh chính), sắp xếp theo `display_order ASC`
- Trả về thống kê rating: `avg_rating`, `review_count`, `rating_distribution` (số review cho mỗi cấp 1-5)

Truy vấn SQL gợi ý:

SQL

-- Query chính: product + store + category

SELECT

```
p.product_id, p.product_name, p.slug, p.description,  
p.price, p.stock_quantity, p.sold_quantity, p.sku,  
p.status, p.created_at, p.updated_at,  
c.category_id, c.category_name, c.slug AS category_slug,  
cp.category_id AS parent_category_id, cp.category_name AS parent_category_n  
s.store_id, s.store_name, s.slug AS store_slug, s.logo_url AS store_logo,  
s.contact_phone AS store_phone, s.province AS store_province,  
s.total_products AS store_total_products
```

FROM products p

INNER JOIN stores s ON p.store_id = s.store_id

INNER JOIN categories c ON p.category_id = c.category_id

LEFT JOIN categories cp ON c.parent_category_id = cp.category_id

WHERE p.product_id = @id

AND p.deleted_at IS NULL

AND p.status = 'ACTIVE'

AND s.deleted_at IS NULL

AND s.status = 'ACTIVE';

-- Query ảnh

SELECT image_url, alt_text, display_order, is_primary

FROM product_images

WHERE product_id = @id

ORDER BY display_order ASC, is_primary DESC;

-- Thông kê rating

SELECT

ROUND(AVG(CAST(rating AS FLOAT)), 1) AS avg_rating,

COUNT(*) AS review_count,

SUM(CASE WHEN rating = 5 THEN 1 ELSE 0 END) AS rating_5,

SUM(CASE WHEN rating = 4 THEN 1 ELSE 0 END) AS rating_4,

SUM(CASE WHEN rating = 3 THEN 1 ELSE 0 END) AS rating_3,

SUM(CASE WHEN rating = 2 THEN 1 ELSE 0 END) AS rating_2,

SUM(CASE WHEN rating = 1 THEN 1 ELSE 0 END) AS rating_1

FROM reviews

WHERE product_id = @id AND status = 'VISIBLE';

Phản hồi thành công (200):

json

```
{
  "success": true,
  "data": {
    "product_id": 1,
    "product_name": "Áo thun basic trắng",
    "slug": "ao-thun-basic-trang",
    "sku": "ATB-WHITE-001",
    "description": "Áo thun cotton 100%...",
    "price": 150000,
    "stock_quantity": 50,
    "sold_quantity": 23,
    "status": "ACTIVE",
    "created_at": "2024-01-15T08:30:00.000Z",
    "updated_at": "2024-01-20T10:00:00.000Z",
    "images": [
      {
        "image_url": "https://...",
        "alt_text": "Áo thun trắng mặt trước",
        "display_order": 0,
        "is_primary": true
      },
      {
        "image_url": "https://...",
        "alt_text": "Áo thun trắng mặt sau",
        "display_order": 1,
        "is_primary": false
      }
    ],
    "category": {
      "category_id": 2,
      "category_name": "Áo",
      "slug": "ao",
      "parent_category": {
        "category_id": 1,
        "category_name": "Thời trang"
      }
    },
    "store": {
      "store_id": 1,
      "store_name": "Shop Thời Trang An",
      "slug": "shop-thoi-trang-an",
      "store_logo": "https://...",
      "contact_phone": "0901234567",
```

```
    "province": "TP. Hồ Chí Minh",
    "total_products": 45
  },
  "rating_summary": {
    "avg_rating": 4.5,
    "review_count": 12,
    "distribution": {
      "5": 7,
      "4": 3,
      "3": 1,
      "2": 1,
      "1": 0
    }
  }
}
```

Phản hồi lỗi (404):

```
json
{
  "success": false,
  "message": "Sản phẩm không tồn tại hoặc đã bị ẩn."
}
```

API #20 — Nhận đánh giá sản phẩm

Cánh đồng	Giá trị
Phương pháp	LÂY
Điểm cuối	<code>/api/v1/products/:id/reviews</code>
Xác thực	Không bắt buộc (Công khai)
Mô tả	Danh sách đánh giá của 1 sản phẩm, có phân trang

Các thông số đường dẫn:

id	BIGINT	product_id
----	--------	------------

Tham số truy vấn:

page	INT	Trang hiện tại (default: 1)
limit	INT	Số review mỗi trang (default: 10, max: 50)
rating	INT	Lọc theo điểm đánh giá (1-5), nếu không truyền thì lấy tất cả
sort_by	STRING	'created_at' 'rating' (default: 'created_at')
sort_order	STRING	'ASC' 'DESC' (default: 'DESC')

Quy tắc kinh doanh:

- Kiểm tra (product_id) Sự tồn tại và (status = 'ACTIVE') → (deleted_at IS NULL) nếu không, trả 404
- Chỉ lấy đánh giá có (status = 'VISIBLE')
- Người đánh giá thông tin: chỉ lấy (full_name) (không hiển thị email, điện thoại)
- (avatar_url) của người đánh giá có thể NULL → mặc định hình đại diện xử lý tự động ở giao diện người dùng
- (comment) có thể NULL (người dùng lựa chọn không để lại nhận xét, chỉ dấu chấm)

Truy vấn SQL gợi ý:

SQL

```
-- Kiểm tra sản phẩm tồn tại
SELECT product_id FROM products
WHERE product_id = @id AND status = 'ACTIVE' AND deleted_at IS NULL;

-- Lấy danh sách review
SELECT
    r.review_id,
    r.rating,
    r.comment,
    r.created_at,
    u.user_id AS reviewer_id,
    u.full_name AS reviewer_name,
    u.avatar_url AS reviewer_avatar
FROM reviews r
INNER JOIN users u ON r.customer_id = u.user_id
WHERE r.product_id = @id
    AND r.status = 'VISIBLE'
    -- AND r.rating = @rating (nếu filter theo rating)
ORDER BY r.created_at DESC
OFFSET (@page - 1) * @limit ROWS
FETCH NEXT @limit ROWS ONLY;

-- Đếm tổng
SELECT COUNT(*) AS total FROM reviews
WHERE product_id = @id AND status = 'VISIBLE'
-- AND r.rating = @rating
```

Phản hồi thành công (200):

json

```
{
  "success": true,
  "data": {
    "product_id": 1,
    "reviews": [
      {
        "review_id": 5,
        "rating": 5,
        "comment": "Sản phẩm đúng mô tả, giao hàng nhanh!",
        "created_at": "2024-02-01T14:30:00.000Z",
        "reviewer": {
          "user_id": 12,
          "full_name": "Lê Văn Cường",
          "avatar_url": "https://..."
        }
      },
      {
        "review_id": 3,
        "rating": 4,
        "comment": null,
        "created_at": "2024-01-28T09:15:00.000Z",
        "reviewer": {
          "user_id": 8,
          "full_name": "Phạm Thị Dung",
          "avatar_url": null
        }
      }
    ],
    "pagination": {
      "current_page": 1,
      "per_page": 10,
      "total_items": 12,
      "total_pages": 2
    }
  }
}
```

BƯỚC 4: YÊU CẦU TRIỂN KHAI

Sau khi đọc mã hiện tại và mẫu rõ ràng, thực hiện:

1. **Tạo/cập nhật tuyến tập** (theo tên và các vị trí khác của tuyến đường đang được sử dụng đúng):

```
GET /api/v1/products          → List Products
GET /api/v1/products/:id      → Get Product Detail
GET /api/v1/products/:id/reviews → Get Product Reviews
```

2. **Tạo bộ điều khiển** với quy ước hiện hành, xử lý chính xác:

- Parse sẽ xác thực các tham số truy vấn.
- Gọi dịch vụ/mô hình tương ứng
- Trả lời về định dạng phản hồi
- Xử lý lỗi (đang sử dụng try/catch hoặc trình bao bọc không đồng bộ)

3. **Tạo dịch vụ/mô hình** với logic:

- Xây dựng câu lệnh SQL cho bộ lọc (dịch thuật tấn công SQL injection, truy vấn tham số hóa)
- Chạy các bài hát truy vấn khi có thể (Promise.all)
- Trả về định dạng dữ liệu

4. **Xác thực đầu vào** theo cách hiện tại (Joi / express-validator / manual):

- `page`, `limit` phải là số dương
- `min_price`, `max_price` không âm
- `sort_by` trong `sort_order` danh sách trắng
- `rating` GIỮA 1 VÀ 5
- `product_id` là số nguyên tố

5. **Đăng ký tuyến** vào tập tin chỉ mục bộ định tuyến (theo đúng cách hiện tại)

6. **Không cần xác thực phần mềm trung gian** vì đây là API Công khai

BƯỚC 5: KIỂM TRA SAU KHI VIẾT

Sau khi xem xong, tự kiểm tra:

- ☐ 3 endpoint đều hoạt động với dữ liệu từ DB
- ☐ Filter `keyword`, `category_id`, `store_id`, `min_price`, `max_price` hoạt động đúng
- ☐ Phân trang trả về đúng `total_items`, `total_pages`
- ☐ Sản phẩm không `ACTIVE` hoặc `deleted` → 404
- ☐ `avg_rating` và `review_count` đúng
- ☐ Ảnh sắp xếp theo `display_order`

- ☐ `rating_distribution` đúng (1→2→3→4→5)
 - ☐ Truy vấn được tham số hóa (không sử dụng bộ lọc nối chuỗi)
 - ☐ Định dạng phản hồi ít nhất so với các API đã viết trước đó
-

⚠ Ý QUAN TRỌNG

- **KHÔNG** tự động đặt quy ước mới — phải theo mẫu đang có trong cơ sở mã
- **KHÔNG** thay đổi các tệp đã viết trước đó — thêm mới hoặc đăng ký tuyên
- Nếu có file `response.helper.js` hoặc `ApiResponse` lớp → dùng nó, không viết lại
- Nếu dự án sử dụng trình bao bọc `async/await` + `try-catch` → sử dụng trình bao bọc đúng
- If project use TypeScript → write type full
- SQL Server: use `OFFSET ... FETCH NEXT ... ROWS ONLY` (không dùng `LIMIT` MySQL)
- SQL Server: sử dụng truy vấn được tham số hóa `@param_name` (không dùng `?` như MySQL)